

Structured Computer Architecture
Chapter 2.10: Automata
Second-Order Systems (2-OS)
An Order-Based Approach

John Glossner and Gheorghe M. Ștefan

Based on the textbook

September 14, 2025

Outline

Learning Objectives

By the end of this chapter, you will be able to:

- ▶ Understand second-order systems (2-OS) and the second feedback loop
- ▶ Analyze finite state machines and their behavior
- ▶ Design control automata for digital systems
- ▶ Distinguish between simple and complex circuits
- ▶ Understand the historical development of automata theory
- ▶ Apply automata concepts to processor design

Evolution from 1-OS to 2-OS

- ▶ **1-OS Limitation:** Partial autonomy only
 - ▶ State maintenance outside input signal range
 - ▶ No autonomous behavior evolution
 - ▶ Reactive to external stimuli
- ▶ **2-OS Innovation:** Second feedback loop
 - ▶ Increases system autonomy
 - ▶ Enables autonomous behavior
 - ▶ Output evolution without input variation
 - ▶ Self-directed state transitions
- ▶ **Key Achievement:**
 - ▶ Behavior autonomy obtained
 - ▶ System can evolve independently
 - ▶ Foundation for intelligent systems

Main Components of 2-OS

- ▶ **Control Automaton:**

- ▶ Manages system behavior
- ▶ Implements state machine logic
- ▶ Controls operation sequences

- ▶ **Program Counter (PC):**

- ▶ Tracks instruction sequence
- ▶ Provides autonomous progression
- ▶ Enables program execution

- ▶ **Registers with Arithmetic & Logic Unit (RALU):**

- ▶ Data processing capability
- ▶ Computational operations
- ▶ State transformation logic

Ancient Origins

▶ **Mythological Beginnings:**

- ▶ **Talos:** Bronze automaton from Greek mythology
- ▶ Self-operating machines in ancient stories
- ▶ Concept of artificial beings

▶ **Early Engineering:**

- ▶ **Hero of Alexandria (10-70 AD):** Pneumatic and hydraulic devices
- ▶ **Su Song (1020-1101 AD):** Water-powered astronomical clock
- ▶ Mechanical figurines and automated systems

▶ **Renaissance Innovations:**

- ▶ **Leonardo da Vinci (1452-1519):** Mechanical knights
- ▶ **Al-Jazari (1136-1206 AD):** Sophisticated water clocks
- ▶ Programmable features in mechanical systems

Mathematical Foundations

► Computational Pioneers:

- **Pascal (17th century)**: Mechanical calculator
- **Jacquard Loom (1804)**: Pioneered programmability
- **Babbage & Lovelace**: Analytical Engine concept

► Theoretical Breakthroughs:

- **Turing (1936)**: Turing Machine model
- **von Neumann**: Self-replicating automata
- **Chomsky**: Formal language classification

► Modern Applications:

- **Shannon**: Boolean circuits foundation
- **Conway**: Game of Life cellular automata
- AI, robotics, and formal verification

Generic Automaton Definition

Formal Definition: Mathematical model of computation

- ▶ Set of states
- ▶ Input alphabet
- ▶ Transition function
- ▶ Initial state
- ▶ Accepting states

Two Main Categories:

- ▶ **Small & Complex:** Finite automata with intricate logic
- ▶ **Large & Simple:** Functional automata with regular patterns

Computational Power:

- ▶ Language recognition capability

Circuit Complexity Measures

▶ **Gate Size (GS):**

- ▶ Total number of CMOS transistor pairs
- ▶ Estimate of silicon area required
- ▶ Hardware implementation cost

▶ **Area Size (AS):**

- ▶ Actual silicon area dimension
- ▶ Production cost factor
- ▶ Yield considerations

▶ **Algorithmic Complexity (C):**

- ▶ Minimal symbols needed for definition
- ▶ Description length in HDL
- ▶ Behavioral complexity measure

Simple vs. Complex Circuits

Simple Circuits:

- ▶ $C_{simple} \ll S_{simple}$
- ▶ Complexity much smaller than size
- ▶ Usually $C_{simple} \in O(1)$
- ▶ Regular, repetitive patterns

Examples:

- ▶ Adder arrays
- ▶ Memory arrays
- ▶ Prefix computation circuits
- ▶ Regular interconnection patterns

Complex Circuits:

- ▶ $C_{complex} \sim S_{complex}$
- ▶ Complexity similar to size
- ▶ Irregular, random-like patterns
- ▶ Difficult to describe concisely

Examples:

- ▶ Random logic circuits
- ▶ Irregular control logic
- ▶ Custom optimization circuits
- ▶ Application-specific designs

Finite Automaton Fundamentals

- ▶ **Formal Definition:** $FA = (Q, \Sigma, \delta, q_0, F)$

- ▶ Q : Finite set of states
- ▶ Σ : Input alphabet
- ▶ δ : Transition function $Q \times \Sigma \rightarrow Q$
- ▶ q_0 : Initial state
- ▶ F : Set of accepting states

- ▶ **Operation Principle:**

- ▶ Start in initial state q_0
- ▶ Read input symbols sequentially
- ▶ Transition according to δ function
- ▶ Accept if final state is in F

- ▶ **Language Recognition:**

- ▶ Regular languages (Type 3)
- ▶ Pattern matching capability
- ▶ Lexical analysis applications

State Transition Diagrams

Graphical Representation:

- ▶ States as circles/nodes
- ▶ Transitions as labeled arrows
- ▶ Initial state marked with arrow
- ▶ Accepting states double-circled

Design Process:

1. Identify required states
2. Define input conditions
3. Specify state transitions
4. Mark accepting states
5. Verify completeness

Analysis Techniques:

- ▶ State reachability

Moore vs. Mealy Machines

Moore Machine:

- ▶ Outputs depend only on current state
- ▶ $\lambda : Q \rightarrow \Gamma$
- ▶ Output changes with state transitions
- ▶ Synchronous output behavior

Advantages:

- ▶ Simpler timing analysis
- ▶ Glitch-free outputs
- ▶ Easier to design
- ▶ More predictable behavior

Mealy Machine:

- ▶ Outputs depend on state and input
- ▶ $\lambda : Q \times \Sigma \rightarrow \Gamma$
- ▶ Faster response to inputs
- ▶ More compact representation

Advantages:

- ▶ Fewer states required
- ▶ Immediate output response
- ▶ More efficient implementation
- ▶ Better for reactive systems

Control Unit Design

- ▶ **Purpose:** Coordinate system operations
 - ▶ Sequence control signals
 - ▶ Manage data flow
 - ▶ Implement instruction execution
 - ▶ Handle exceptions and interrupts
- ▶ **Design Approaches:**
 - ▶ **Hardwired Control:** Fixed logic implementation
 - ▶ **Microprogrammed Control:** Stored program approach
 - ▶ **Hybrid Control:** Combination of both methods
- ▶ **Implementation Considerations:**
 - ▶ Speed vs. flexibility trade-offs
 - ▶ Area and power constraints
 - ▶ Testability and verification
 - ▶ Modification and upgrade capability

Program Counter (PC) Automaton

- ▶ **Function:** Autonomous instruction sequencing
 - ▶ Maintains current instruction address
 - ▶ Provides sequential progression
 - ▶ Handles branches and jumps
 - ▶ Enables program execution flow
- ▶ **Operations:**
 - ▶ **Increment:** Normal sequential execution
 - ▶ **Load:** Branch and jump operations
 - ▶ **Reset:** System initialization
 - ▶ **Hold:** Wait states and stalls
- ▶ **Advanced Features:**
 - ▶ Branch prediction support
 - ▶ Exception handling
 - ▶ Multi-threading support
 - ▶ Debug and trace capabilities

Beyond Regular Languages

▶ **Regular Language Limitation:**

- ▶ Cannot recognize $\{1^n 0^n | n > 0\}$
- ▶ Finite memory constraint
- ▶ Need for counting capability

▶ **Counter-Expanded Automaton (CEA):**

- ▶ Finite automaton + reversible counter
- ▶ Counter provides zero flag to automaton
- ▶ Enables context-free language recognition
- ▶ 3-OS system (FA + Counter in loop)

▶ **Enhanced Capabilities:**

- ▶ Balanced parentheses checking
- ▶ Nested structure recognition
- ▶ Stack-like behavior simulation
- ▶ Context-free grammar parsing

CEA Example: $a^n b^n$ Language

- ▶ **Problem:** Recognize strings with equal numbers of *as* and *bs*
 - ▶ Context-free language
 - ▶ Requires counting mechanism
 - ▶ Cannot be solved by finite automaton alone

- ▶ **CEA Solution:**
 - ▶ States: $\{q_0, q_1, q_2, q_3, q_4\}$
 - ▶ Counter operations: $\{nop, up, down\}$
 - ▶ Zero flag from counter
 - ▶ Delimiter symbol *e* for well-formed strings

- ▶ **Operation Sequence:**
 1. Count *as* by incrementing counter
 2. Switch to *b* processing mode
 3. Count *bs* by decrementing counter
 4. Accept if counter reaches zero

Hardware Implementation

▶ **State Encoding:**

- ▶ Binary encoding: Minimal flip-flops
- ▶ One-hot encoding: Fast decoding
- ▶ Gray code: Reduced switching
- ▶ Custom encoding: Application-specific

▶ **Logic Implementation:**

- ▶ Combinational next-state logic
- ▶ State register (flip-flops)
- ▶ Output logic (Moore/Mealy)
- ▶ Clock and reset distribution

▶ **Optimization Techniques:**

- ▶ State minimization
- ▶ Logic synthesis
- ▶ Timing optimization
- ▶ Power reduction

Design Verification

▶ **Functional Verification:**

- ▶ State coverage analysis
- ▶ Transition coverage testing
- ▶ Input sequence validation
- ▶ Output correctness checking

▶ **Formal Methods:**

- ▶ Model checking techniques
- ▶ Temporal logic verification
- ▶ Equivalence checking
- ▶ Property verification

▶ **Testing Strategies:**

- ▶ Directed test cases
- ▶ Random test generation
- ▶ Corner case analysis
- ▶ Stress testing

Processor Control Units

▶ **Instruction Decode:**

- ▶ Opcode interpretation
- ▶ Addressing mode handling
- ▶ Resource allocation
- ▶ Pipeline control

▶ **Execution Control:**

- ▶ ALU operation sequencing
- ▶ Memory access coordination
- ▶ Register file management
- ▶ Exception handling

▶ **System Control:**

- ▶ Interrupt processing
- ▶ Power management
- ▶ Debug support
- ▶ Performance monitoring

Memory Controllers

▶ **DRAM Control:**

- ▶ Refresh cycle management
- ▶ Row/column addressing
- ▶ Timing constraint enforcement
- ▶ Bank management

▶ **Cache Controllers:**

- ▶ Hit/miss detection
- ▶ Replacement policy implementation
- ▶ Coherency protocol management
- ▶ Prefetch control

▶ **I/O Controllers:**

- ▶ Protocol state machines
- ▶ Data transfer management
- ▶ Error detection and correction
- ▶ Flow control

Hierarchical State Machines

- ▶ **Motivation:** Managing complexity

- ▶ Large state spaces
- ▶ Nested behaviors
- ▶ Modular design
- ▶ Reusability

- ▶ **Statecharts:**

- ▶ Hierarchical states
- ▶ Concurrent regions
- ▶ History states
- ▶ Broadcast communication

- ▶ **Benefits:**

- ▶ Reduced complexity
- ▶ Better maintainability
- ▶ Improved readability
- ▶ Easier verification

Concurrent and Parallel Automata

▶ **Parallel Composition:**

- ▶ Multiple automata running simultaneously
- ▶ Independent state evolution
- ▶ Synchronization points
- ▶ Communication mechanisms

▶ **Synchronization Methods:**

- ▶ Shared signals
- ▶ Message passing
- ▶ Rendezvous points
- ▶ Barrier synchronization

▶ **Applications:**

- ▶ Multi-core processors
- ▶ Distributed systems
- ▶ Protocol implementations
- ▶ Parallel algorithms

Key Concepts Review

▶ **Second-Order Systems (2-OS):**

- ▶ Second feedback loop enables autonomy
- ▶ Behavior evolution without input changes
- ▶ Foundation for intelligent systems

▶ **Automata Theory:**

- ▶ Mathematical models of computation
- ▶ State-based behavior description
- ▶ Language recognition capabilities

▶ **Implementation Aspects:**

- ▶ Hardware realization techniques
- ▶ Optimization strategies
- ▶ Verification methodologies

Design Principles

▶ **Complexity Management:**

- ▶ Simple vs. complex circuit classification
- ▶ Hierarchical design approaches
- ▶ Modular implementation strategies

▶ **Performance Considerations:**

- ▶ State encoding optimization
- ▶ Timing constraint satisfaction
- ▶ Power and area trade-offs

▶ **Verification and Testing:**

- ▶ Formal verification methods
- ▶ Coverage-driven testing
- ▶ Model checking techniques

Connection to Higher-Order Systems

▶ **Building Blocks for 3-OS (Processors):**

- ▶ Control unit implementation
- ▶ Instruction sequencing logic
- ▶ Exception handling mechanisms

▶ **System-Level Integration:**

- ▶ Memory controller design
- ▶ I/O protocol implementation
- ▶ Inter-component communication

▶ **Advanced Applications:**

- ▶ Multi-core coordination
- ▶ Distributed system control
- ▶ Adaptive system behavior

Questions and Discussion

Questions?

Discussion Topics:

- ▶ How does the second feedback loop enable autonomous behavior?
- ▶ What are the trade-offs between Moore and Mealy machines?
- ▶ How do counter-expanded automata extend computational power?
- ▶ What role do automata play in modern processor design?

Next: Chapter 2.11 - Processors: Third-Order Systems (3-OS)